BUSINESSINTELLIGENCE.AI

KPI Storytelling Engine

From “what changed” to “what to do” — an AI system design for explaining, diagnosing, and acting on metric shifts

System Design Document

A production-grade architecture for automated root-cause analysis and natural-language narrative generation over structured and unstructured business data — with explicit treatment of statistical noise, causal inference, confidence calibration, and human-in-the-loop escalation.

PREPARED FOR	Hackathon Submission — Track: Business Intelligence & Applied AI
DOCUMENT TYPE	Technical Architecture & Design Rationale
SCOPE	Detection → Diagnosis → Narrative → Recommendation → Feedback

CONTENTS

Table of Contents

SECTION 1

Problem Framing & Design Goals

Dashboards are good at reporting **what** happened and poor at explaining **why**, and they never explain **what to do about it**. When a regional revenue metric drops 8%, an analyst today manually pulls transaction data, checks for outliers, cross-references marketing spend, reads support tickets, and skims competitor news — a process that takes one to several days and does not scale across the hundreds of KPIs a mid-size business tracks. The KPI Storytelling Engine (KSE) is designed to compress this loop from days to minutes by automating detection, diagnosis, explanation, and recommendation, while remaining honest about uncertainty.

Design goals

- **Statistically defensible:** every flagged change must survive a formal significance test against a seasonally-aware baseline before being surfaced.
- **Causally grounded, not just correlational:** root causes are ranked using structured decomposition and causal-inference techniques, not p-hacked correlations.
- **Evidence-linked narrative:** every sentence the system generates must be traceable to a specific data point, so leaders can verify, not just trust.
- **Actionable by design:** outputs terminate in a ranked set of recommended actions with estimated impact and confidence, not just a diagnosis.
- **Honest under ambiguity:** when evidence is thin or conflicting, the system says so explicitly and escalates to a human rather than guessing confidently.
- **Multi-modal by construction:** structured metrics and unstructured text (tickets, news, call transcripts, emails) are fused into one evidence graph, not analyzed in silos.

SECTION 2

System Architecture Overview

The KSE is organized as five sequential layers sitting on top of a unified ingestion pipeline, with a persistent feedback loop from analysts back into the detection and attribution models. Each layer is independently scalable and independently testable, which matters for both engineering robustness and for demoing individual components in a hackathon setting.

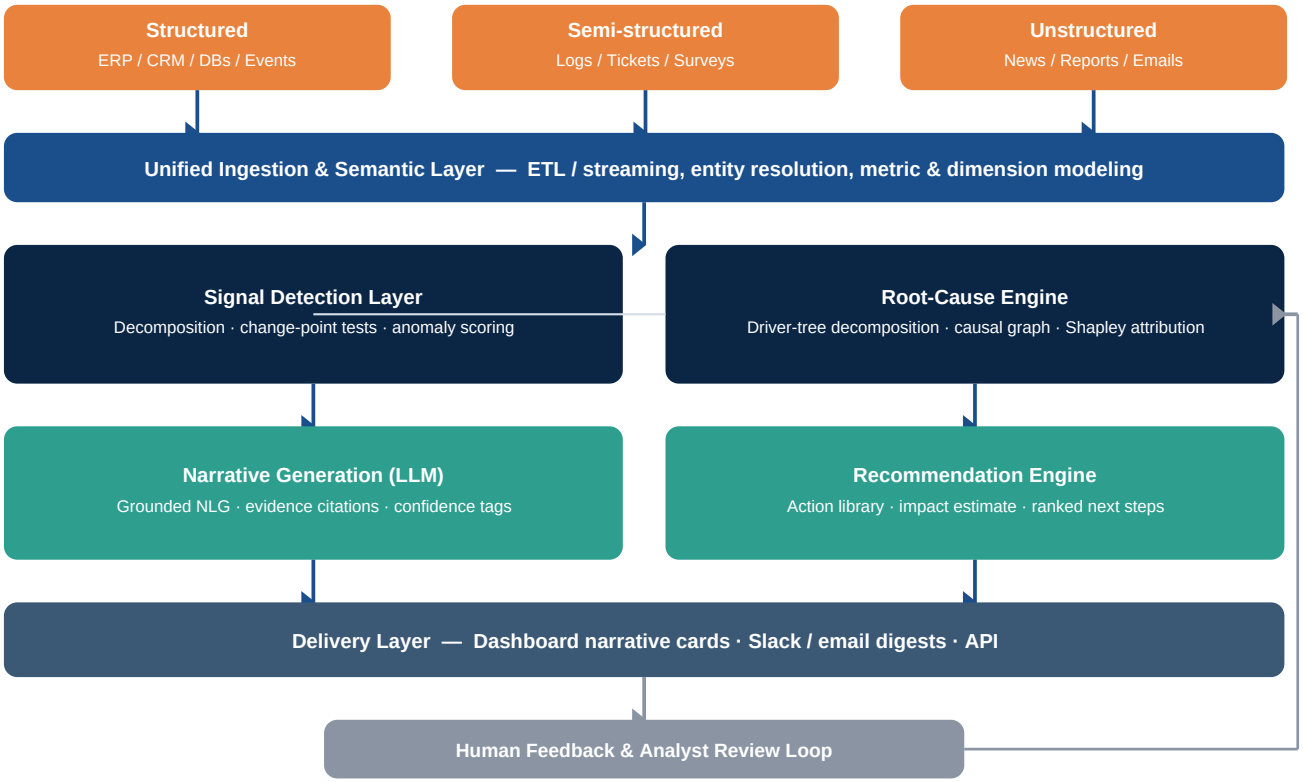

Figure 1. End-to-end architecture — ingestion, detection, root-cause analysis, narrative generation, recommendation, delivery, and the human feedback loop that retrains upstream models.

Why this decomposition

- **Separation of detection and explanation.** A statistical engine decides *whether* a change is real; a separate causal engine decides *why*. Coupling these two responsibilities is the most common failure mode in “AI insight” tools, which tend to explain noise.
- **LLM is a translator, not an analyst.** The language model only narrates conclusions that were already computed numerically upstream. It is explicitly barred from inventing causes — it selects from and phrases a pre-computed, ranked evidence set.
- **Everything is swappable.** The signal-detection algorithm, the causal model, and the LLM provider can each be replaced without touching the others, which is what “industry standard” modularity looks like in production BI systems.

SECTION 3

Layer 1 — Unified Data Ingestion

KPI storytelling requires fusing three data modalities that most BI stacks keep separate. The ingestion layer's job is to bring them into one time-aligned, entity-resolved evidence store before any analysis happens.

Data Type	Examples	Ingestion Approach
Structured	Revenue, orders, pricing, inventory, marketing spend, web analytics, CRM stage changes	Batch + streaming ELT (CDC from OLTP, event streams via Kafka/Kinesis) into a metrics warehouse; conformed dimensional model (star schema) so every KPI shares consistent time, region, product, and channel keys
Semi-structured	Support tickets, survey responses, app logs, error/incident logs	Schema-on-read ingestion into a document store; structured fields (timestamp, category, region) extracted directly, free-text fields queued for NLP enrichment
Unstructured	News articles, competitor pricing pages, earnings calls, internal emails, Slack threads, call transcripts	Scheduled crawlers / connector APIs → chunking → embedding → vector index; entity and event extraction (NER + relation extraction) to tag documents with company, region, product, and date

Semantic layer

All three modalities are mapped onto a shared **metric & entity ontology** — a metrics-layer definition (e.g. dbt semantic layer or a custom metric store) that fixes, once, what “revenue,” “region,” and “active customer” mean across every source. This is the single most important robustness decision in the system: without it, the root-cause engine will confidently explain a metric drift that is actually a definitional mismatch between two systems.

- **Entity resolution:** customer, product, and region identifiers are resolved to canonical IDs across CRM, billing, and support systems before any correlation is computed.
- **Time alignment:** all sources are bucketed to a common grain (daily by default) with explicit timezone and fiscal-calendar handling.
- **Lineage tracking:** every ingested fact carries source, extraction method, and confidence metadata, which later powers the evidence citations in the generated narrative.

SECTION 4

Layer 2 — Signal Detection: Separating Change from Noise

This is the layer that answers the question the prompt raises directly: *how does the engine know an 8% move is real and not noise?* The approach is a pipeline of three checks, each of which a movement must pass before it is escalated to root-cause analysis.

Step 1 — Decompose before you judge

Raw metric values are decomposed using STL (Seasonal-Trend decomposition using Loess) or a Bayesian structural time series model into trend, seasonality, and residual components. An 8% headline drop that is 90% explained by a known seasonal pattern (e.g. a predictable post-holiday dip) is filtered out before it ever reaches an analyst.

Step 2 — Test, don't eyeball

- **Statistical process control:** control charts (e.g. EWMA, CUSUM) flag a residual only when it crosses a threshold calibrated from the metric's own historical variance, so noisy KPIs and stable KPIs get different sensitivity automatically.
- **Change-point detection:** Bayesian online change-point detection or PELT identifies whether the series has genuinely shifted regime, versus a single volatile data point.
- **Multiple-comparisons correction:** because the system scans thousands of metric × dimension combinations (region × product × channel), raw p-values are corrected (Benjamini-Hochberg FDR control) so the system isn't flooded with false positives from sheer search volume.

Step 3 — Materiality gate

Statistical significance is necessary but not sufficient — a statistically real 0.3% move in a low-priority metric should not generate a narrative. A materiality score combines statistical confidence with business weight (revenue exposure, OKR linkage, historical volatility-adjusted magnitude) to decide what actually surfaces to a human.

Check	Question it answers	Typical technique
Decomposition	Is this explained by known seasonality/trend?	STL, Prophet, BSTS
Significance	Is the residual outside normal variance?	EWMA / CUSUM control limits, change-point detection
Multiplicity control	Are we crying wolf due to scanning many slices?	Benjamini-Hochberg FDR correction
Materiality	Does this matter to the business?	Weighted score: revenue exposure × confidence × OKR linkage

SECTION 5

Layer 3 — Root-Cause Analysis: From Correlation to Explanation

Once a movement clears the significance gate, the system must move from “what correlates with this drop” to “what plausibly caused it.” Three complementary techniques are combined rather than relying on any single one, because each has different failure modes.

5.1 Structured driver-tree decomposition

The metric is decomposed algebraically into its known drivers — e.g. Revenue = Active Customers × Avg Order Value × Purchase Frequency, further broken down by region, channel, and product line. This immediately isolates whether an 8% revenue drop is a volume story or a price story, before any machine-learning inference is needed. This step alone resolves a large share of real cases and is fully explainable by construction.

5.2 Contribution attribution (Shapley-style)

Where the driver tree does not cleanly localize the cause (e.g. the drop is spread across many small segments), Shapley-value decomposition allocates the total change in the metric across candidate dimensions (region, channel, customer segment, cohort) so that contributions sum exactly to the observed change — avoiding the double-counting that ad-hoc correlation slicing produces.

5.3 Causal graph reasoning over structured + unstructured evidence

A causal graph (DAG) encodes known business relationships (e.g. “marketing spend → traffic → conversions → revenue”; “competitor price cut → our conversion rate”). Candidate causes surfaced from unstructured sources (a spike in support tickets mentioning “checkout error,” a competitor promotion detected in scraped pricing pages, a negative product review cluster) are mapped onto nodes in this graph. Techniques such as Granger causality tests and difference-in-differences (comparing affected vs. unaffected but similar segments) are used to test whether a candidate cause precedes and is proportionate to the effect, rather than just co-occurring with it.

Why this avoids the correlation trap

- **Temporal precedence is enforced:** a candidate cause must be shown to precede the metric shift, not just coincide with it.
- **Counterfactual comparison:** difference-in-differences against a comparable, unaffected segment (e.g. a similar region without the checkout bug) tests whether the effect is specific to the hypothesized cause.
- **Magnitude consistency check:** the estimated size of the candidate cause's effect must be consistent with the observed magnitude of the KPI move — a 2% traffic dip cannot be presented as the sole explanation for an 8% revenue drop.
- **Ranked, not singular:** the engine always outputs a ranked list of hypotheses with confidence scores rather than one “the cause is X” statement, reflecting genuine uncertainty in observational data.

SECTION 6

Layer 4 — Narrative Generation Engine

The narrative layer's only job is to translate a structured evidence object — significance test results, driver-tree decomposition, ranked causal hypotheses with confidence scores, and supporting text snippets — into a natural-language story a business leader can read in under a minute. It is deliberately constrained so it cannot introduce facts that were not computed upstream.

Grounded generation pattern

- The evidence object is serialized into a structured prompt; the LLM is instructed to use only the facts provided and to cite the source field for every causal claim it makes.
- A retrieval-augmented generation (RAG) step pulls the two or three most relevant unstructured evidence snippets (a support ticket excerpt, a news headline) to quote or closely paraphrase alongside the quantitative findings.
- A post-generation validator checks every numeric claim in the generated text against the evidence object programmatically (regex / entity match) and rejects or regenerates any sentence containing an unsupported number.
- Each narrative is templated into: *Headline → What changed (with magnitude and confidence) → Why it likely happened (ranked causes with evidence) → What we're not sure about → What to do next*.

This "LLM-as-narrator, not analyst" pattern is the key robustness decision for this layer: it bounds hallucination risk to phrasing rather than fact invention, which is what makes the system auditable and trustworthy enough for executive-facing output.

SECTION 7

Layer 5 — Recommendation Engine

Diagnosis without action is still just a smarter dashboard. The recommendation engine converts each ranked root cause into a small set of concrete next steps, each with an estimated impact and effort so leaders can prioritize.

Root Cause Category	Example Recommended Actions	Impact Estimation
Operational (e.g. checkout bug)	Escalate to engineering with affected segment and ticket volume trend; suggest hotfix priority	Recovered revenue ≈ pre-incident run-rate – current run-rate, scoped to affected segment
Competitive (e.g. price cut)	Recommend targeted promotion or price-match test in affected region; flag for pricing team review	Elasticity-based estimate using historical price-test data for that segment
Demand-side (e.g. seasonal/marketing)	Reallocate spend toward underperforming channel; adjust forecast baseline	Marginal ROAS from the marketing mix model, applied to proposed reallocation
Data / definitional issue	Flag pipeline or metric definition for data-engineering review before further analysis	N/A — recommendation is to fix instrumentation, not the business

Recommendations are drawn from a curated, versioned **action library** mapped to cause categories rather than freely generated, so every suggestion is one a domain expert has pre-approved as sensible — the LLM's role here is to select and rank the right action template and populate it with the specific numbers from this incident, not to invent business strategy from scratch.

SECTION 8

Handling Ambiguity & Low-Confidence Cases

Real business data is frequently ambiguous — multiple plausible causes, sparse unstructured evidence, or a change too recent to have accumulated enough signal. The system is designed to make ambiguity visible rather than papering over it with false confidence.

Signal	System Behavior
Two or more causal hypotheses with similar confidence scores	Narrative presents both explicitly (“most likely explained by A or B”) with the evidence for each, instead of arbitrarily picking one
Confidence score below the actionability threshold	Output is labeled “under investigation”; recommendation defaults to “monitor + gather more data” rather than a false action
Conflicting evidence across sources (e.g. tickets say one thing, sales data another)	Both threads are surfaced side-by-side with their respective evidence rather than silently reconciled
Insufficient historical data for the segment (new product / region)	System explicitly states the limitation and widens the comparison group instead of forcing a narrow, unreliable baseline
Any case above a defined uncertainty threshold	Routed to the human feedback queue for analyst review; analyst's resolution is captured and fed back to retrain the confidence model

This is a deliberate product principle, not just an engineering fallback: a system that is wrong confidently is more dangerous than one that is silent. Every generated narrative therefore carries a visible confidence tier (High / Moderate / Low-Needs Review), and low-confidence outputs are never auto-sent to executive channels without analyst sign-off.

SECTION 9

Worked Example — 8% Regional Revenue Drop

Walking the prompt's own scenario through the pipeline end-to-end:

1. Detection. STL decomposition shows the drop is not explained by seasonality; EWMA control chart flags the residual as a genuine regime shift ($p < 0.01$ after FDR correction). Materiality score is high given the region's revenue share → escalated.

2. Driver-tree. Decomposition shows Active Customers flat, Average Order Value flat, Purchase Frequency down 9% — isolating this to a repeat-purchase problem, not an acquisition or pricing problem.

3. Unstructured evidence. NLP enrichment surfaces a 3x spike in support tickets tagged ‘checkout error’ in that region starting two days before the metric shift, plus zero relevant competitor or news signal.

4. Causal check. Difference-in-differences against a comparable region without the bug shows purchase frequency there unchanged over the same window — supporting checkout errors as the primary driver over alternative hypotheses.

5. Narrative output. “Region X revenue is down 8% this week, driven almost entirely by a 9% drop in repeat purchase frequency, not acquisition or pricing. This coincides with a 3x spike in checkout-error tickets starting Aug 11. Confidence: High.”

6. Recommendation. Escalate to engineering with ticket volume and affected user segment; estimated recoverable revenue ≈ pre-incident run-rate once resolved; suggest customer-success outreach to affected accounts in the interim.

SECTION 10

Technology Stack

A stack chosen for maturity and horizontal scalability rather than novelty, reflecting what production BI/AI systems run today.

Layer	Representative Technologies
Ingestion / streaming	Kafka or Kinesis for events; Fivetran / Airbyte for batch CDC; Airflow or Dagster for orchestration
Storage	Cloud warehouse (Snowflake / BigQuery / Redshift) for structured data; object store + vector DB (pgvector, Pinecone, or Weaviate) for unstructured embeddings
Semantic / metrics layer	dbt semantic layer or Cube.dev to enforce one shared definition of every KPI and dimension
Statistical / ML detection	Python: statsmodels, Prophet, ruptures (change-point detection), scikit-learn; Shapley attribution via SHAP
Causal inference	DoWhy / EconML for causal graph estimation and diff-in-diff
NLP enrichment	spaCy / transformer NER for entity and event extraction from tickets, news, transcripts
Narrative generation	LLM via API with structured grounding prompts and RAG; output validated programmatically before delivery
Orchestration of the AI pipeline	A workflow/agent framework (e.g. LangGraph or a custom DAG runner) to chain detection → attribution → narration → validation as auditable steps, not a single opaque agent call
Delivery	Dashboard embed (narrative cards next to charts), Slack/email digest, and a REST API for downstream consumption
Observability	Data quality monitoring (Great Expectations / Monte Carlo-style checks), model performance dashboards, prompt/response logging

SECTION 11

Scalability, Reliability & Governance

Scalability

- **Decoupled, queue-based orchestration:** detection, attribution, and narration run as independent, horizontally-scalable services connected by a message queue, so a spike in flagged metrics doesn't block the ingestion pipeline.
- **Incremental computation:** statistical models update incrementally on new data rather than recomputing full history on every run, keeping cost roughly linear in new-data volume, not total history size.
- **Tiered analysis depth:** lightweight significance testing runs on every metric × dimension slice continuously; the expensive causal-graph and LLM narration steps only run on the small subset that clears the materiality gate — this is what makes covering thousands of KPIs economically viable.
- **Caching of embeddings and evidence retrieval** so repeated queries against the same unstructured corpus don't re-embed or re-search unnecessarily.

Reliability

- Every pipeline stage is idempotent and checkpointed so partial failures can be safely retried without duplicating alerts.
- The LLM narration step has a deterministic, template-based fallback: if the model call fails or the output fails validation, the system still delivers the structured evidence (numbers, ranked causes) without the prose layer, rather than failing silently.
- SLA-tiered processing: business-critical KPIs (e.g. company-wide revenue) get priority queue placement and tighter latency targets than long-tail metrics.

Governance & trust

- **Full lineage and auditability:** every generated sentence links back to the exact query, model version, and data snapshot that produced it, satisfying enterprise audit requirements.
- **Access control aligned to data sensitivity:** narrative outputs respect the same row/column-level security as the underlying warehouse, so a regional manager cannot see another region's diagnosis.
- **PII handling:** unstructured sources (emails, transcripts) are scrubbed/redacted for PII before embedding or inclusion in prompts sent to any external LLM API.
- **Model risk management:** narration and recommendation prompts, and their validators, are versioned and regression-tested against a labeled set of historical incidents before each deployment, similar to how a fraud or credit model would be governed.

SECTION 12

Evaluation Framework & Success Metrics

A system like this must be evaluated on both statistical correctness and business usefulness — optimizing only for one produces either a noisy alert system or a confident-sounding but unreliable one.

Dimension	Metric	Target Behavior
Detection quality	Precision / recall of flagged anomalies vs. analyst-labeled ground truth	High precision prioritized — false alarms erode trust faster than missed ones
Attribution quality	Top-1 / top-3 root-cause accuracy against historically resolved incidents	Ranked causes should contain the eventual confirmed cause within top 3, most of the time
Narrative faithfulness	Fraction of generated numeric claims that pass the automated grounding validator	Target near-100% — any ungrounded claim is a hard failure, not a quality nuance
Calibration	Reliability diagram: stated confidence vs. observed correctness rate	A “High confidence” call should be right roughly as often as its stated confidence implies
Business impact	Time-to-diagnosis, analyst hours saved, % of recommendations acted on	Reduction from days to minutes; recommendations adopted, not ignored

SECTION 13

Risks, Limitations & Roadmap

Known limitations

- Causal inference on observational business data can never fully replace controlled experiments; the system should recommend A/B tests to confirm high-stakes causal claims before large resource commitments.
- Unstructured-data coverage is only as good as the connectors available — a cause that exists purely in an un-integrated data source (e.g. a sales rep's private notes) will not be found.
- LLM narration quality depends on the grounding validator; this validator itself needs ongoing maintenance as new metric types and phrasing patterns are added.

Phased rollout

Phase	Scope
Phase 1 (MVP / hackathon)	Structured-data-only detection + driver-tree decomposition + templated narrative for a single KPI family (e.g. revenue)
Phase 2	Add unstructured ingestion (tickets, news) and RAG-grounded narration; introduce confidence tiers and the human feedback loop
Phase 3	Full causal-graph reasoning, Shapley attribution across arbitrary dimension combinations, recommendation-impact backtesting
Phase 4	Self-improving loop: analyst resolutions automatically retrain detection thresholds and causal-hypothesis ranking

Closing note

The differentiator of this design versus a generic “LLM summarizes your dashboard” approach is that the language model is never the source of an insight — it is the last, tightly-constrained step in a pipeline where every prior stage is statistically and causally accountable. That is what makes the output something a business leader can act on rather than something they have to double-check.